Arten

Edge Object Detection, Counting & Attributes

Author: Sujal M H · Solo participant

Namma AI Hackathon 2026 · Problem Statement 2 — AI for Safer Communities

---

Abstract

Arten is an offline edge-AI proof of concept that detects, counts and attributes pedestrians and vehicles in a short street clip or image set. A compact YOLO11n INT8 ExecuTorch model runs on a Raspberry Pi 4 or 5 CPU; a source-built ARMv8.0 runtime covers Pi 3/4 when the prebuilt wheel cannot run. The pipeline writes per-frame counts, on-device direction, dwell and relative-speed attributes, optional coarse vehicle colour, and annotated frames. A tiled mode runs native-resolution crops and merges the boxes, trading latency for recall on dense scenes. Optional pose and segmentation models add posture, rider tags and a road/sidewalk surface per detection. On the committed clip it processes 150 frames, peaks at 11 people in one frame and produces 31 indicative person tracks plus two background vehicles. A 50-run benchmark on the Pi 5 measures **136.73 ms p50 / 7.31 FPS** with **435.6 MB** process RSS; Arm's card reports **134.10 ms / 7.46 FPS** and **38.90% mAP@0.5:0.95**.

1. Problem and application

Road traffic injuries are the leading cause of death for people aged 5–29 worldwide (WHO). India recorded **1,68,491 road deaths in 2022**; pedestrians were **19.5%** of the fatalities and two-wheeler riders **44.5%** (MoRTH).

A school, ward office or resident group does not need another surveillance platform. It needs a cheap, repeatable answer to one question: *how many people cross here, when, and alongside how much traffic?* That answer is usually locked behind expensive cameras, proprietary dashboards or continuous cloud connectivity.

Application. A school records three short clips at the same gate: morning drop-off, afternoon dismissal and a quiet hour. It runs them locally on a Raspberry Pi. The output is a count table, movement attributes and a few annotated frames. A high pedestrian/vehicle overlap at dismissal supports an on-site review for a crossing guard, a zebra crossing or a no-parking zone. The clip supplies evidence for that review; it does not estimate near-misses or crash risk.

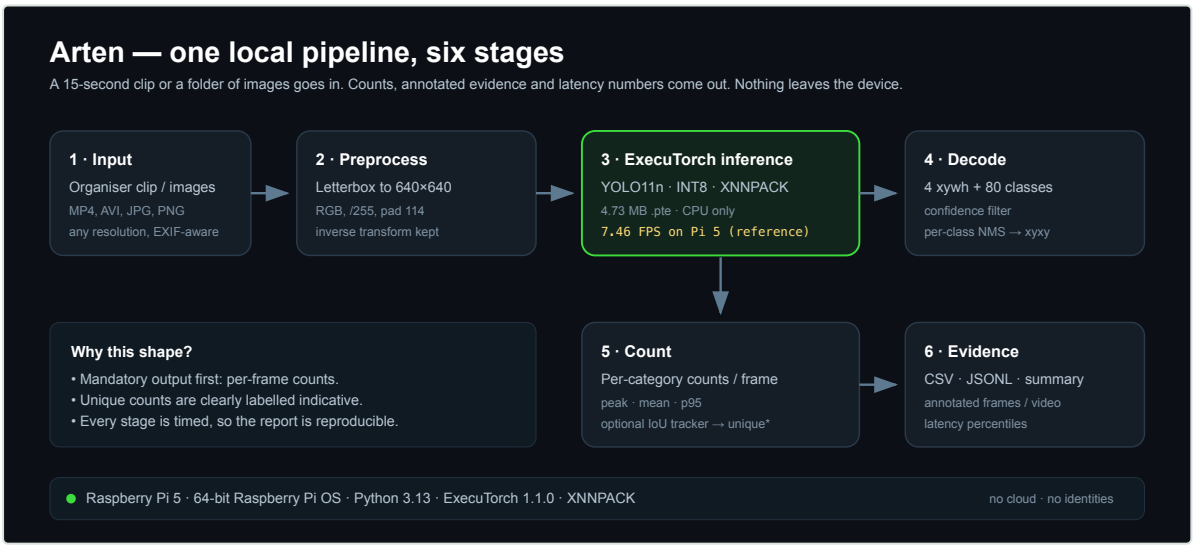
2. Approach

The design makes six choices:

1. **Use an edge artifact.** Arm's pre-optimized YOLO11n INT8 `.pte` is small enough for a Pi and verified on the target. A larger YOLOv8s `.pte` is an optional accuracy upgrade; the CLI resolves either model from its checksum.
2. **Make counts the first-class output.** Per-frame counts come directly from detections, independent of tracking quality.

3. **Add attributes on the same tracker.** Direction, dwell and relative speed reuse the existing tracker. Optional colour is classical HSV. No second model.
4. **Calibrate and tile when needed.** The default confidence is 0.2, tuned on real street frames. A 2×2 or 3×3 tiled pass recovers small objects on dense frames at 4× or 9× the cost.
5. **Ship evidence.** Every run writes counts, detections, attribute rows, latency percentiles and annotated frames.
6. **Add detail with small support models.** A 2.82 MB top-down pose model and a 5.64 MB road segmenter run on a cadence, adding posture, rider tags and surface without replacing the detector.

Property	Value
Model	YOLO11n (default); YOLOv8s accuracy option
Quantization	INT8 static PTQ; stem conv and detection head FP32
Runtime (Pi 4/5)	ExecuTorch 1.1.0, XNNPACK + KleidiAI
Runtime (Pi 3/4 fallback)	source-built ExecuTorch 1.1.0, torch 2.9.0, XNNPACK
Tiled mode	2×2 or 3×3 native crops, ~4× / ~9× latency
Support models	Lite-HRNet18 pose (Apache-2.0) · DDRNet23-slim surface (MIT)
Pi 5 reference	**134.10 ms p50 · 7.46 FPS · 59.17 MB peak**
COCO accuracy	**38.90% mAP@0.5:0.95 · 54.66% mAP@0.5**
Model size	**4.73 MB**



Pipeline

The pipeline reads video with OpenCV or images with PIL and EXIF transpose, letterboxes to 640×640 with the 114-grey pad, runs the ExecuTorch `forward` method, decodes the 4 xywh + 80 class outputs, applies per-class NumPy NMS and maps boxes back to the original image. It then counts each category, updates the tracker, attaches attributes and writes the evidence bundle.

Attribute	Definition	Limit
Direction	Sign of the smoothed bbox velocity: left, right, up, down, stationary	Image plane, not a compass
Relative speed	Velocity magnitude in pixels/second	Uncalibrated; compare within one clip
Dwell	Time a confirmed track stays visible	Visibility duration, not queue time
Vehicle colour	HSV histogram over the central crop	Heuristic; sensitive to light, glass and shadows
Zone co-presence	Counts inside a manual polygon	Descriptive, not near-miss or crash risk

The layer is deliberately non-learned. It adds explainable movement context without face, plate, age, gender or identity attributes.

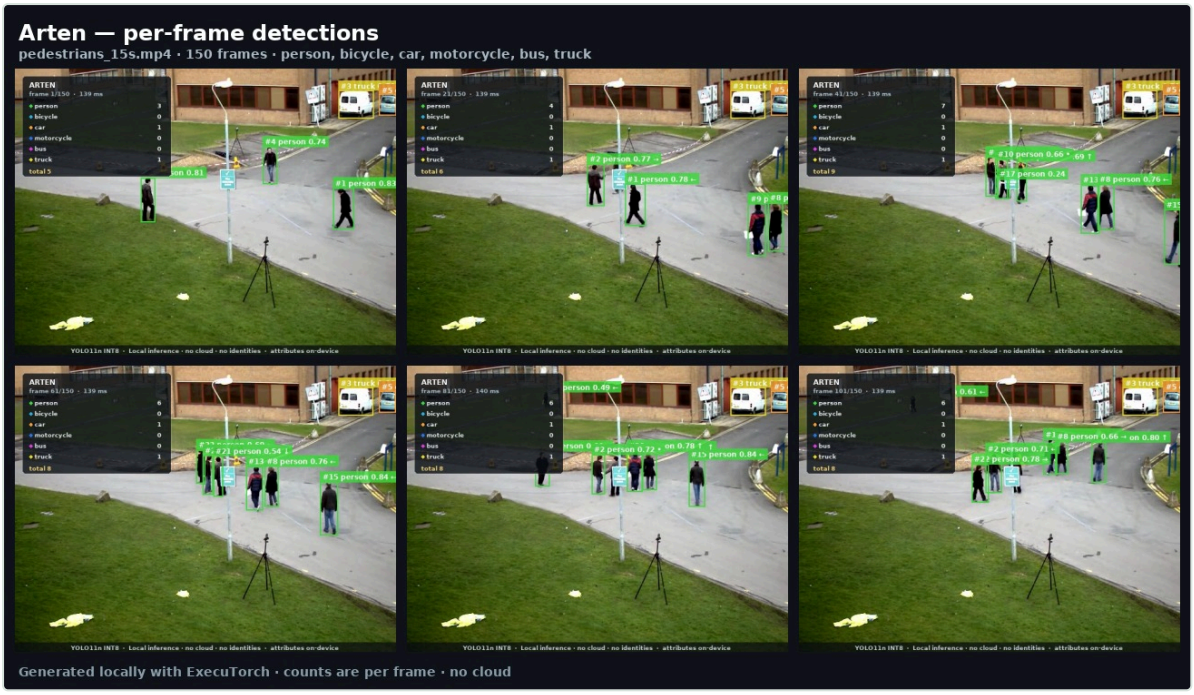
3. Results

3.1 Pedestrian clip

Measured on the Raspberry Pi 5 over 150 frames at the default confidence 0.2.

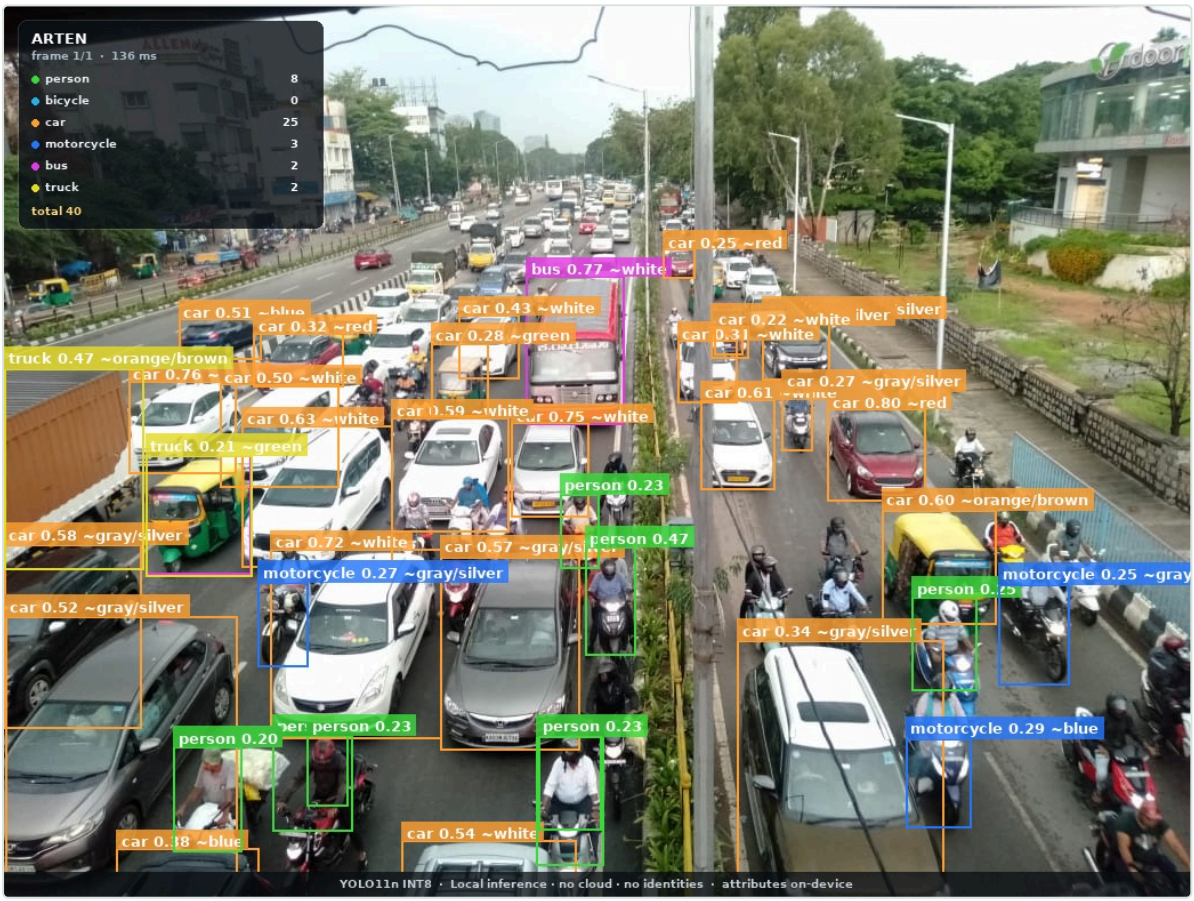
Category	Peak/frame	Mean/frame	p95	Unique (indicative)
person	11	5.92	8	31
bicycle	0	0.00	0	0
car	1	1.00	1	1
motorcycle	0	0.00	0	0
bus	0	0.00	0	0
truck	1	1.00	1	1

The 31 person tracks split **13 left, 6 right, 6 stationary, 3 down, 3 up**, with **mean dwell 2.83 s** and **mean relative speed 38.69 px/s**. The car (blue) and truck (white) are background vehicles and stationary. The unique count is indicative: tracker IDs switch when people cross or leave frame. The per-frame counts are the stable output.



Pedestrian contact sheet

3.2 Dense Bengaluru traffic — stress test



Dense traffic

At the calibrated 0.2 threshold the Pi 5 frame yields **8 people, 25 cars, 3 motorcycles, 2 buses and 2 trucks**. The colour heuristic labels the 25 cars as **11 white, 7 gray/silver, 3 red, 2 blue, 1 green, 1**

orange/brown; the buses are green/white and the trucks green/orange. The labels are visibly imperfect, so they are a demo of derived attributes, not a reliable colour measurement.

A manual count of the same frame finds roughly **30–40 people, 45–55 cars and 25–30 motorcycles**, so the full-frame pass recovers about half. The scene also has three auto-rickshaws, which COCO cannot label.

Tiled inference changes the picture. On the Pi 5, a 2×2 pass with YOLO11n finds **26 people, 40 cars and 13 motorcycles** at **547.65 ms/frame**; YOLOv8s finds **27 people, 41 cars and 19 motorcycles** at **935.22 ms/frame**. A 3×3 pass reaches **30/43/14** (YOLO11n, 1.23 s) and **30/44/21** (YOLOv8s, 2.12 s). No retraining is involved: the model runs on native-resolution crops and the boxes are merged. The trade-off is latency, and bus/truck counts become less reliable because large vehicles produce partial boxes across crops.

3.3 Latency

Measured on the Raspberry Pi 5 with the prebuilt ExecuTorch 1.1.0 + torch 2.10 stack. The runtime starts 10 threads; Arm's model-card numbers use one thread.

Stage	Mean ms	p50 ms	p95 ms	FPS from mean
Model inference	138.85	138.84	139.24	7.20
Preprocess	3.01	3.04	3.12	332.55
Decode + NMS	3.42	3.41	3.48	292.38
Frame read	0.84	0.66	1.54	1188.83
Annotate/save	14.64	14.43	16.86	68.31
Pipeline total	**162.20**	**162.05**	**165.17**	**6.17**

A standalone 50-run benchmark measures **136.73 ms p50 / 7.31 FPS** with **435.6 MB** process RSS, within 2% of Arm's **134.10 ms / 7.46 FPS** card. Tiled inference costs about 4× per 2×2 grid: 547.65 ms (YOLO11n) and 935.22 ms (YOLOv8s); 3×3 costs 1.23 s and 2.12 s.

Arm's YOLOv8s card reports 457.03 ms with **one thread**. The same model here measures **233.81 ms / 4.28 FPS** with the default runtime threading, so the card number is not directly comparable. On the ARMv8.0 Pi 3B+ build, YOLO11n measured 1295 ms at a 600 MHz under-voltage cap; that is compatibility evidence, not the target performance.

Support models add detail on a cadence. Lite-HRNet18 costs **142 ms per person crop** and DDRNet **86 ms per frame**. With pose every 10 frames (three people) and segmentation every 30, the 150-frame clip measures **190.6 ms mean / 5.25 FPS** with YOLO11n and **291.1 ms / 3.43 FPS** with YOLOv8s. Support frames cost about 454 ms; the other nine frames are detector-only.

3.4 Mandatory deliverables

Requirement	Artifact
Process a short clip or image set on ExecuTorch + Pi	Committed sample clip; `arten run`; Pi setup script
Detect a category and show per-frame counts	`counts.csv`; `summary.txt`; annotated HUD
Category + counts + two sample outputs	Pedestrian contact sheet; annotated traffic/tiled images; surface and posture attributes
Report inference time, model and throughput	`summary.json`; `results/benchmark_pi5.json`; §3.3
Explain the issue and one application in the short pitch	`docs/pitch-2min.md`; §1

4. Limitations and responsible AI

- **COCO is not India.** Auto-rickshaws, hand carts and many local vehicle types are not COCO classes. At 0.2 the dense frame finds 25 cars and 3 motorcycles, against a manual count of roughly 45–55 and 25–30; tiling raises that to 41/19 (2×2) or 44/21 (3×3). The three auto-rickshaws remain unlabelled.
- **Tiling is a latency trade.** A 2×2 pass costs about 4× and a 3×3 pass about 9×. Bus and truck counts become less reliable because large vehicles produce partial boxes across crops.
- **Support details are sampled and indicative.** Surface and posture are not trained risk classifiers. Pose runs on the largest people only, and segmentation is a coarse 90×120 mask with no grass class; grass is often labelled road. Use them for context, not enforcement.
- **No ground-truth mAP on organiser data.** The report cites COCO reference accuracy and manual counts, not a fabricated accuracy on the clip.
- **Attributes are derived, not learned.** Direction is image-plane, speed is uncalibrated pixels/second, colour is a coarse HSV heuristic, and tracker IDs can switch. The 31 person tracks at 0.2 include switches; per-frame counts are the stable output. These attributes support within-clip comparison, not enforcement.
- **Threading changes the comparison.** Arm's model cards use one thread; the Pi 5 runtime here starts ten, so YOLOv8s measures 233.81 ms against a 457.03 ms single-thread card. Compare like-for-like before quoting either.
- **Board coverage.** The Pi 5 numbers are measured. Pi 4 is estimated from the Pi 5 and Pi 3B+ scaling. The Pi 3B+ compatibility run was capped at 600 MHz by under-voltage, so its absolute numbers are not the target performance.
- **Aggregate, not identity.** The system extracts no faces, plates, age, gender or identity fields. Annotated frames still contain the original pixels, so treat them as sensitive evidence and blur them before publication.
- **Licence.** YOLO11n, YOLOv8s and the Arm `.pte` files are AGPL-3.0; this repository is AGPL-3.0 and includes the model notice.

5. Reproduce

```
bash scripts/setup_pi.sh
source .venv/bin/activate
arten doctor
arten run -i data/samples/pedestrians_15s.mp4 --conf 0.2 \
  -o results/pi_run --save-video --estimate-vehicle-colors
arten run -i data/samples/bengaluru_traffic.jpg --conf 0.2 --tile 2 \
  -o results/pi_tiled
arten run -i data/samples/pedestrians_15s.mp4 \
  --pose-model models/lite_hrnet18_int8_xnnpack.pte --pose-every 10 \
  --seg-model models/ddrnet23_slim_int8_xnnpack.pte --seg-every 30 \
  -o results/pi_detail
arten benchmark --input data/samples/bengaluru_traffic.jpg --runs 50
```

`setup_pi.sh` detects the board: Pi 5 uses the prebuilt ARMv8.2 wheels, Pi 3/4 the committed ARMv8.0 runtime. With a `wheelhouse/` directory next to the repo it installs without network (verified: 22 wheels, 156 MB, zero downloads).

Model SHA-256: `154c54b52389219ebd982e1003227c1abbc3f29a99bdad241147b684d3041b96`.

Sample media, model card and licences are documented in `data/README.md` and `models/README.md`. Source: https://github.com/sujalmh/arten-edge-ai.

References

1. WHO. *Road traffic injuries* fact sheet, 2026. https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries
2. MoRTH, Government of India. *Road Accidents in India 2022*, 2023. https://morth.gov.in/backend/documents/uploaded/1755600426_RA_2022_30_Oct.pdf
3. Arm. *YOLO11n INT8 ExecuTorch for Raspberry Pi 5*. https://huggingface.co/Arm/yolo11n-int8-xnnpack-executorch-raspberrypi5
4. Arm. *YOLOv8s INT8 ExecuTorch*. https://huggingface.co/Arm/yolov8s-int8-xnnpack-executorch
5. Ultralytics. *ExecuTorch integration*. https://docs.ultralytics.com/integrations/executorch
6. PyTorch. *ExecuTorch documentation*. https://docs.pytorch.org/executorch/
7. Namma AI Hackathon. https://www.coe-iot.com/namma-ai-hackathon/